

# Strukturalni paterni

U ovom dokumentu navedeni su strukturalni paterni korišteni u modifikaciji trenutnog dijagrama sistema klasa i razlozi za njihov odabir.

## 1. Facade patern - KupoprodajaFacade

### Problem

U sistemu za kupoprodaju i iznajmljivanje artikala, klijentski kod (npr. kontroler u prezentacijskom sloju) koji želi izvršiti operaciju kupovine mora samostalno koordinirati između više klasa: provjeriti dostupnost `Artikal` objekta, kreirati `Transakcija` objekat, eventualno kreirati `Rezervacija` objekat, te validirati `Korisnik` koji vrši transakciju. Ovo znači da klijent mora poznavati internu strukturu svih tih klasa i njihove međusobne zavisnosti, što rezultira čvrstom spregom (tight coupling) između prezentacijskog sloja i poslovne logike.

### Rješenje

Uvedena je klasa `KupoprodajaFacade` koja implementira interfejs `ITransakcijaFacade` i nudi samo dvije jasne, visokonivojske metode prema klijentu: `kupiArtikl()` i `iznajmiArtikl()`. Unutar sebe, Facade interno koordinira sve potrebne korake — provjerava korisnika, provjerava dostupnost artikla, kreira transakciju i po potrebi kreira rezervaciju. Klijent ne zna ništa o tim klasama i ne ovisi direktno o njima, već komunicira isključivo kroz Facade interfejs.

### Zašto je primjenljiv u ovom sistemu

Sistem već razlikuje dva tipa transakcije kroz `TipTransakcijeEnum` (prodaja i iznajmljivanje), što znači da logika unutar `kupujArtikl()` i `iznajmiArtikl()` prirodno postoji u domeni — Facade je samo organizuje na jednom mjestu. Bez Facade paterna, svaki novi klijent (mobilna aplikacija, web interfejs) morao bi sam reimplementirati tu koordinacijsku logiku.

### Konsekvence

Facade smanjuje spregu između slojeva sistema i čini prezentacijski sloj nezavisnim od promjena u poslovnoj logici. Ako se npr. proces kreiranja rezervacije promijeni, mijenja se samo Facade, a ne svaki klijent koji ga koristi. Mana je što Facade može postati tzv. "God object" ako se previše logike akumulira u njemu, pa je važno da samo delegira pozive a ne sadrži kompleksnu logiku sam.

## 2. Proxy patern - ArtiklServisProxy

### Problem

Klasa `Artikal` i operacije nad njom (dodavanje, uređivanje, uklanjanje) nisu dozvoljene svim korisnicima jednako — sistem ima `UlogaEnum` koji razlikuje kupce, prodavače i moderatore, te `aktivan` atribut na `Korisnik` klasi. Međutim, direktan pristup servisnoj logici ne nudi nikakav mehanizam kontrole pristupa. Svaki pozivalac može pozvati bilo koju metodu bez provjere.

### Rješenje

Uveden je interfejs `IArtikalServis` koji definira ugovor za sve operacije nad artiklima. Ovaj interfejs implementiraju dvije klase. `ArtikalServisImpl` je tzv. RealSubject — stvarna implementacija koja izvršava operacije nad `Artikal` objektima. `ArtikalServisProxy` je Proxy koji stoji ispred `ArtikalServisImpl`-a: svaki poziv metode prvo prolazi kroz privatnu metodu `provjeriPristup(korisnik)` koja čita `uloga` i `aktivan` attribute iz `Korisnik` objekta. Tek ako je pristup dozvoljen, Proxy delegira poziv pravoj implementaciji. Klijent uvijek razgovara samo sa `IArtikalServis` interfejsom i ne zna da li koristi Proxy ili direktnu implementaciju.

### Zašto je primjenljiv u ovom sistemu

Sistem već posjeduje sve potrebne elemente za ovaj patern: `UlogaEnum` definira uloge korisnika, `aktivan: boolean` na `Korisnik` klasi omogućava provjeru aktivnosti, a `ZahtjevZaUklanjanje` i `AdminModerator` klase jasno ukazuju da različiti korisnici imaju različita prava nad artiklima. Proxy centralizuje te provjere na jednom mjestu umjesto da se ponavljaju kroz cijelu aplikaciju.

### Konsekvence

Proxy patern omogućava dodavanje kontrole pristupa bez ikakve izmjene u `ArtikalServisImpl` klasi, što je u skladu sa Open/Closed principom — klase su otvorene za proširenje, ali zatvorene za modifikaciju. Klijentski kod se također ne mijenja jer i dalje koristi isti `IArtikalServis` interfejs. Mana je uvođenje dodatnog sloja indirekcije koji blago povećava kompleksnost sistema, ali ta cijena je opravdana sigurnošću i konzistentnošću kontrole pristupa.